

Отчёт по заданию 2.3

Тема: Исследование возможностей клиента для работы с Git на примере командной строки (терминала).

Выполнил: Бебутов Х.

Группа: ИВТ-2

Дата: 26.09.2026

1. Цель работы

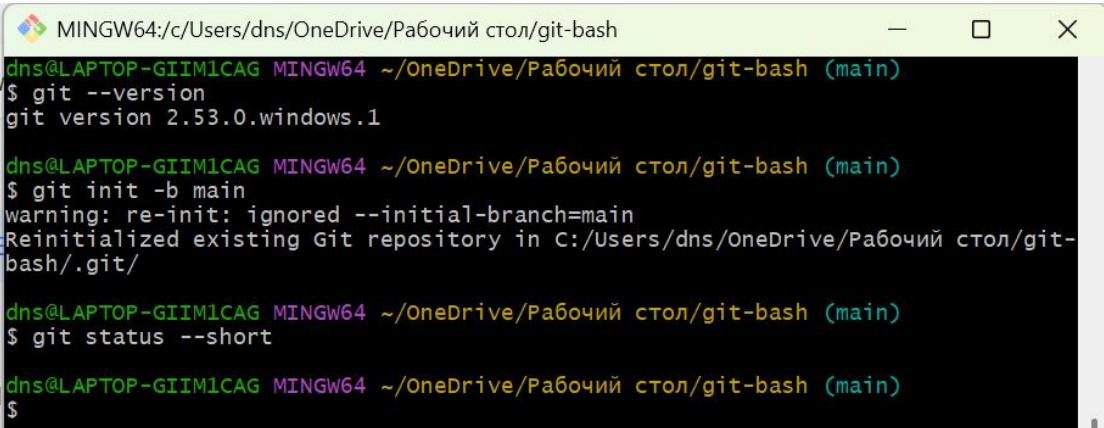
Исследовать возможности одного из клиентов для работы с Git, продемонстрировать выполнение наиболее распространённых команд, а также особенности конкретного инструмента.

В качестве инструмента выбран **Терминал (командная строка)**. Этот вариант не требует установки дополнительного графического клиента, работает на любой операционной системе и даёт полный контроль над командами Git.

Проект содержит README.md с описанием задачи и stats.py с решением задачи: вычисление среднего, минимума и максимума для списка чисел.

1. Создание репозитория и просмотр состояния

```
git --version  
git init -b main  
git status --short
```

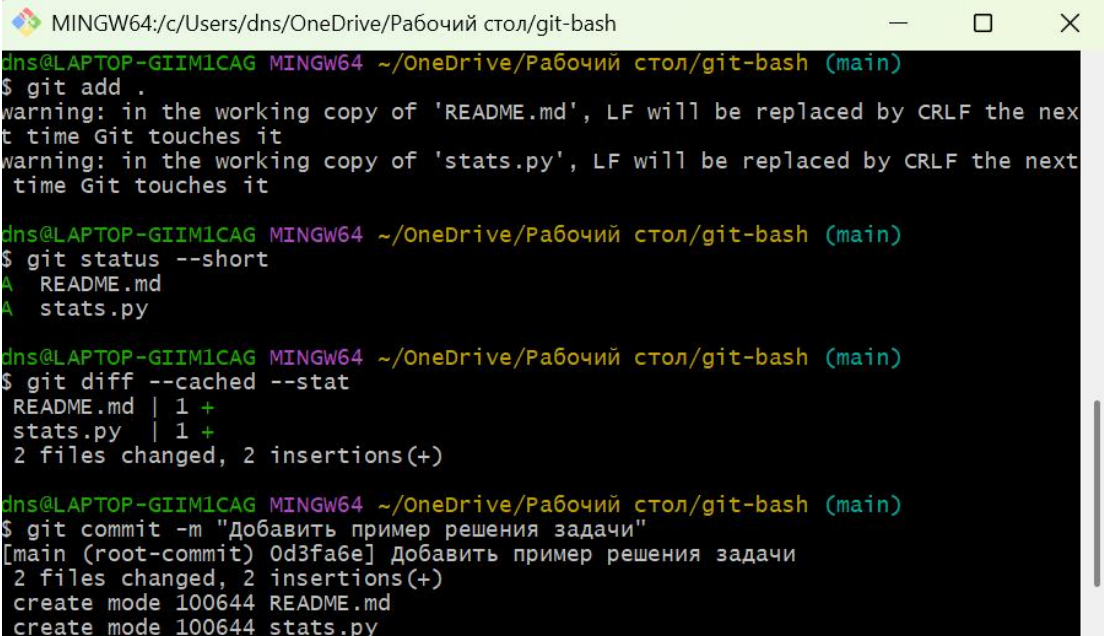


```
MINGW64:/c:/Users/dns/OneDrive/Рабочий стол/git-bash  
dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)  
$ git --version  
git version 2.53.0.windows.1  
  
dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)  
$ git init -b main  
warning: re-init: ignored --initial-branch=main  
Reinitialized existing Git repository in C:/Users/dns/OneDrive/Рабочий стол/git-bash/.git/  
  
dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)  
$ git status --short  
  
dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)  
$
```

Команда `git init -b main` создаёт локальный репозиторий с основной веткой `main`. Обозначение `??` указывает на файлы, которые ещё не отслеживаются Git. В командах настройки автора следует указать собственные данные перед работой в своём репозитории.

2. Добавление и создание коммита

```
git add .
git status --short
git diff --cached --stat
git commit -m "Добавить пример решения задачи"
```



```
MINGW64:/c:/Users/dns/OneDrive/Рабочий стол/git-bash
dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)
$ git add .
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'stats.py', LF will be replaced by CRLF the next time Git touches it

dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)
$ git status --short
A README.md
A stats.py

dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)
$ git diff --cached --stat
README.md | 1 +
stats.py | 1 +
2 files changed, 2 insertions(+)

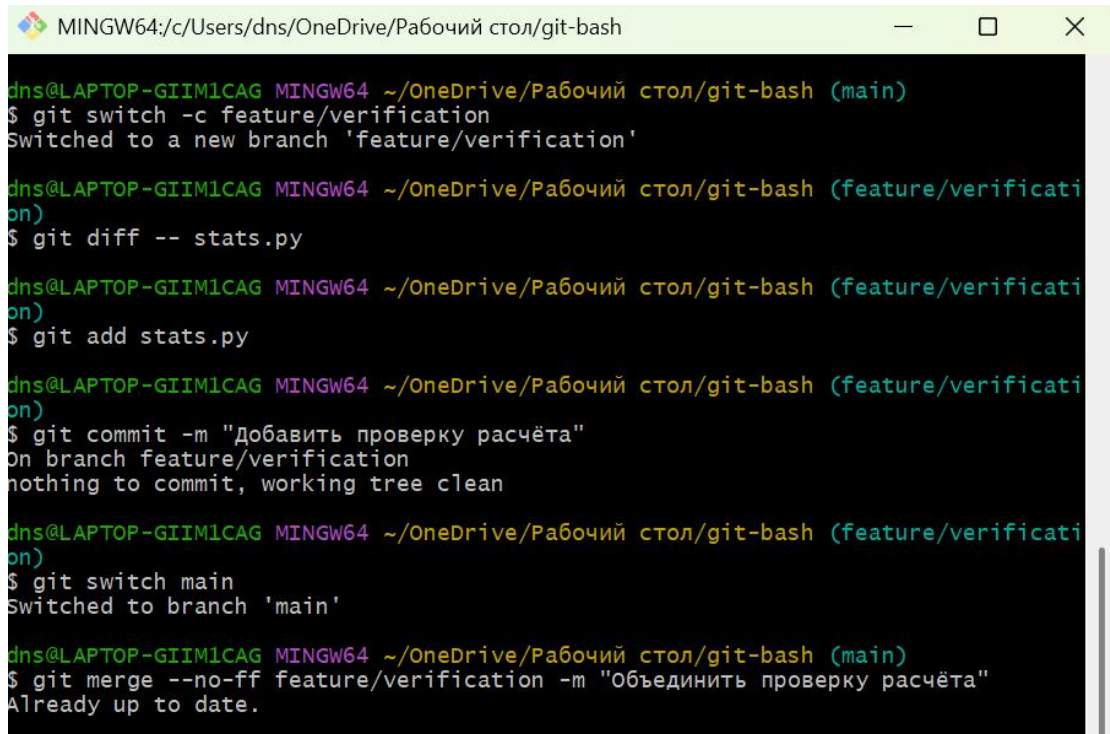
dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)
$ git commit -m "Добавить пример решения задачи"
[main (root-commit) 0d3fa6e] Добавить пример решения задачи
2 files changed, 2 insertions(+)
create mode 100644 README.md
create mode 100644 stats.py
```

Команда `git add .` переносит текущие версии двух файлов в индекс. После этого буква `A` в первом столбце сокращённого статуса означает подготовленные к коммиту новые файлы. `git diff --cached` показывает именно индекс; `git commit` сохраняет подготовленное состояние как коммит с хешем.

3. Изменение в отдельной ветке

```
git switch -c feature/verification
git diff -- stats.py
git add stats.py
```

```
git commit -m "Добавить проверку расчёта"
git switch main
git merge --no-ff feature/verification -m "Объединить проверку расчёта"
```

A screenshot of a terminal window titled "MINGW64:/c/Users/dns/OneDrive/Рабочий стол/git-bash". The terminal shows a sequence of Git commands and their outputs. The user starts in the 'main' branch, switches to a new branch 'feature/verification', creates a file 'stats.py', commits it with the message "Добавить проверку расчёта", switches back to 'main', and finally merges the 'feature/verification' branch into 'main' using the '--no-ff' option. The merge is successful and the terminal shows "Already up to date."

```
dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)
$ git switch -c feature/verification
Switched to a new branch 'feature/verification'

dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (feature/verification)
$ git diff -- stats.py

dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (feature/verification)
$ git add stats.py

dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (feature/verification)
$ git commit -m "Добавить проверку расчёта"
On branch feature/verification
nothing to commit, working tree clean

dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (feature/verification)
$ git switch main
Switched to branch 'main'

dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)
$ git merge --no-ff feature/verification -m "Объединить проверку расчёта"
Already up to date.
```

Команда `git switch -c` создаёт ветку и сразу переходит в неё. `git diff` без `--cached` показывает ещё не подготовленные изменения. После возврата в `main` команда `git merge` с параметром `--no-ff` сохранила отдельный коммит слияния; стратегия `ort` завершилась без конфликта.

4. Восстановление файла

```
git status --short
git restore README.md
git status --short
```

A screenshot of a terminal window with a green title bar. The title bar text is "MINGW64:/c/Users/dns/OneDrive/Рабочий стол/git-bash". The terminal shows a series of commands and their outputs. The prompt is "dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)". The commands are: "git status --short", "git restore README.md", and "git status --short". The output of the first and third commands is empty. The prompt returns after each command.

```
dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)
$ git status --short

dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)
$ git restore README.md

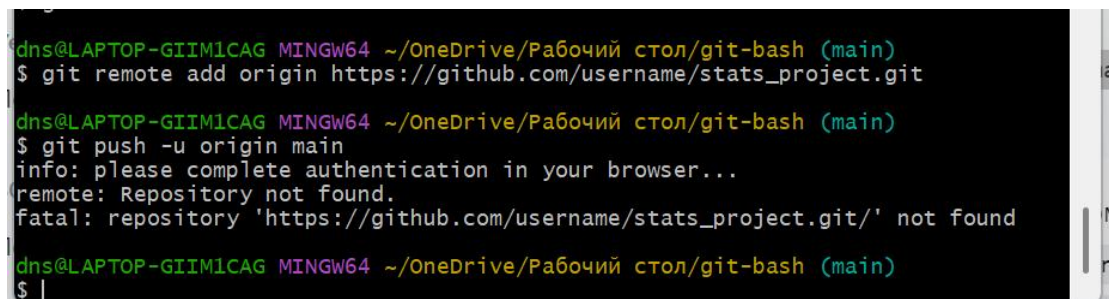
dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)
$ git status --short

dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)
$
```

Для опыта в README.md была дописана строка «Черновик». Команда `git restore` вернула версию из последнего коммита; повторный статус пуст, потому что непроиндексированных изменений не осталось. Этот шаг отменяет именно незакоммиченное изменение.

5. Работа с удалённым репозиторием

```
git remote add origin https://github.com/username/stats_project.git
git push -u origin main
```

A screenshot of a terminal window with a green title bar. The title bar text is "MINGW64:/c/Users/dns/OneDrive/Рабочий стол/git-bash". The terminal shows the commands "git remote add origin https://github.com/username/stats_project.git" and "git push -u origin main". The output of the first command is empty. The output of the second command is "info: please complete authentication in your browser...", "remote: Repository not found.", and "fatal: repository 'https://github.com/username/stats_project.git/' not found". The prompt returns after each command.

```
dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)
$ git remote add origin https://github.com/username/stats_project.git

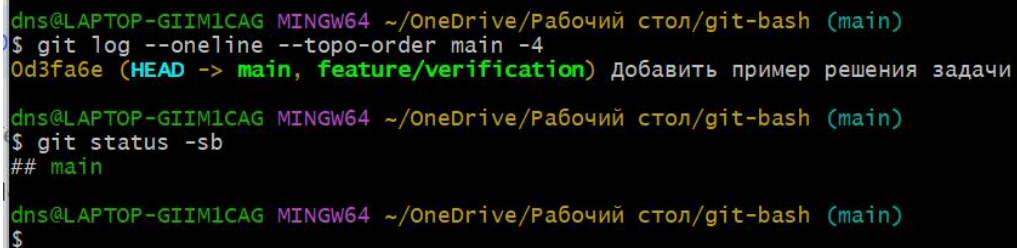
dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)
$ git push -u origin main
info: please complete authentication in your browser...
remote: Repository not found.
fatal: repository 'https://github.com/username/stats_project.git/' not found

dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)
$
```

Команда `git remote add` связывает локальный репозиторий с удалённым. Команда `git push` отправляет изменения на удалённый сервер. Ключ `-u` устанавливает связь между локальной и удалённой ветками, чтобы в дальнейшем можно было использовать просто `git push`.

6. История и итоговое состояние

```
git log --oneline --topo-order main -4  
git status -sb
```



```
dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)  
> $ git log --oneline --topo-order main -4  
0d3fa6e (HEAD -> main, feature/verification) добавить пример решения задачи  
  
dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)  
$ git status -sb  
## main  
  
dns@LAPTOP-GIIM1CAG MINGW64 ~/OneDrive/Рабочий стол/git-bash (main)  
$
```

История содержит начальный коммит, изменение в отдельной ветке, коммит слияния и правку из второй копии.

Вывод

Терминал даёт явный контроль над индексом, веткой и origin. Если origin уже задан, повторная команда `git remote add origin` завершится ошибкой. Выполнены: создание репозитория, коммиты, ветвление, слияние, отмена локальной правки и отправка изменений на удалённый сервер. Результаты подтверждены выводом команд репозитория.